TEAM 1

CS 315

PROJECT 2

Ayşegül Yıldız 22002591 Sec:1

Duru Solakoğlu 22102720 Sec:1

Erkan Can Arslan 22103948 Sec:1

Name of Our Programming Language: COSTANZA

CONTENT TABLE

1. **Complete BNF Description of COSTANZA**

   1.1. Non-Trivial Tokens
   1.2. Program Definition
   1.3. Non-Terminals

2. Description of Nontrivial Tokens

   2.1. Comment
   2.2. Identifiers
   2.3. Literals
   2.4. Reserved Words
   2.5. Operators & Symbols

3. Description of Nonterminals

4. Language Criteria
   4.1. Readability
   4.2. Writability
   4.3. Reliability

1. Complete BNF Description of COSTANZA:

1.1. Non-Trivial Tokens

<RUN>	::= **run**
<LB>	::= **{**
<RB>	::= **}**
<SLB>	::= **[**
<SRB>	::= **]**
<END_LINE>	::= **;**
<ASSIGN_OP>	::= **<-**
<LOW_ABC>	::= **a | b | c| d | e | f| g | h | ı | j | k | l | m | n | o | p | r | s | t |u | v | y | z**
<UP_ABC>	::= **A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P | R | S | T | U | V | Y | Z**
<DIGIT>	::= **0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9**
<SWITCH>	::= **s1 | s2| s3 | s4 | s5 | s6 | s7 | s8 | s9 | s10**
<PLUS_SIGN>	::= **+**
<MINUS_SIGN>	::= **-**
<DIVID_SIGN>	::= **/**
<MULT_SIGN>	::= *****
<MOD_SIGN>	::= **%**
<POW_SIGN>	::= **^**
<IF>	::= **if**
<ELSE>	::= **else**
<ELIF>	::= **elif**

<LP> ::= **(**

<RP> ::= **)**

<OR_SIGN> ::= |

<AND_SIGN> ::= **&**

<NOTEQUAL_SIGN> ::= **not =**

<EQUAL_SIGN> ::= **==**

<LESS_SIGN> ::= **<**

<GREAT_SIGN> ::= **>**

<GREATEQUAL_SIGN> ::= **>=**

<LESSEQUAL_SIGN> ::= **<=**

<START_COMMENT> ::= **<***

<END_COMMENT> ::= ***>**

<FUNCTION_CALL> ::= **>>**

<COMMA> ::= **,**

<INCREMENT> ::= ++

<QUOTATION> ::= **“**

<DECREMENT> ::= **--**

<SEPARATOR> ::= **%%**

<SYMBOLS> ::= **!** | **&** | **?** | \ | **.** |-| _ | | # | **:** | =

<FOR> ::= **for**

<WHILE> ::= **while**

<RETURN> ::= **return**

<FUNCTION> ::= **function>>**

<BOOLEAN> ::= **true** | **false**

<GET_TEMP> ::= **getTemperature**

<GET_HUM> ::= **getHumidity**

<GET_AP> ::=**getAirPressure**

<GET_AQ> ::= **getAirQuality**

<GET_LIGHT> ::= **getLight**

<GET_SOUND> ::= **getSound**

<NOT> ::= **not**

<CONNECT_URL> ::= **connectToURL**

<SWITCH_ON> ::= **switchOn**

<SWITCH_OFF> ::= **switchOff**

<GET_TIME> ::= **getTime**

<SEND_INT> ::= **sendInt**

<RECEIVE_INT> ::= **receiveInt**

<TO> ::= **to**

<FROM> ::= **from**

<PRINT> **::= print**

<URL> ::= **URL**

Primitive Types

<ABC> ::= <LOW_ABC> | <UP_ABC>

<ABC_DIGIT> ::= <ABC> | <DIGIT>

<INT_POS> ::= <DIGIT> | <DIGIT><INT_POS>

<INT_NEG> ::= <MINUS_SIGN> <INT_POS>

<INT> ::= <INT_POS> | <INT_NEG>

<DOUBLE> ::= <INT> <POINT> <INT_POS>

<ID> ::= <LOW_ABC> <RESTOFIT>

<VAR_WORD> ::= <ABC_DIGIT><VAR_WORD> | <ABC_DIGIT>

<RESTOFIT> ::=<RESTOFIT><_> <VAR_WORD>

|<VAR_WORD>

<CMT_WORD> ::= <ABC> | <DIGIT | <SYMBOLS>

<STRING> ::= <QUOTATION> <CMT_WORD> <QUOTATION>

|<QUOTATION> <CMT_WORD>

<QUOTATION><STRING>

<ARRAY_INIT> ::= <LB> <VAR_INITS> <RB> | <LB> <RB>

<VAR_INITS> ::= <VAR_INIT> | <VAR_INITS> <COMMA>

<VAR_INIT>

<VAR_INIT> ::= <INT> | <ID> | <ARRAY_INIT>

Program Definition

<program> ::= <RUN><LB><stmt_list><RB><SEPARATOR>

<function_declarations>

1.3. Non-Terminals

Statements

<stmt_list> ::= <stmt> | <stmt> <stmt_list> | <NEW_LINE>

<stmt> ::= <matched> | <unmatched>

<elif_stmts> ::= <elif_stmt>|<elif_stmt><elif_stmts>|<NEW_LINE>

<elif_stmt> ::= <ELIF><LP><logic_expr><RP><LB><stmt_list>

<RB>

<matched> ::= <IF> <LP><logic_expr><RP><LB> <stmt_list> <RB>

<elif_stmts> <ELSE><LB><matched><RB>

| <nonconditional_stmt>

<unmatched> ::= <IF> <LP> <logic_expr><RP><LB> <stmt_list> <RB>

<elif_stmts>

| <IF> <LP> <logic_expr><RP> <LB><stmt_list> <RB>
<elif_stmts> <ELSE><LB> <unmatched> <RB>

<nonconditional_stmt> ::= <prim_functs>

| <comment>

| <assignment>

| <arithmetic_op>

| <loops>

|<return_stmt>

|<function_caller>

|<print>

User Defined Function

<function_declarations> ::= | <function_declaration>
<function_declarations>

<function_declaration> ::= <FUNCTION><ID> <LP> <param_list> <RP>
<LB> <stmt_list><RB>

<function_caller> ::= <FUNCTION_CALL><ID><LP><param_list>
<RP> | <FUNCTION_CALL> <prim_functs>

<param_list> ::= |<term>| <term> <COMMA>
<param_list>

Loops

<loops> ::= <for_loop> | <while_loop>

<for_loop> ::= <FOR> <LP> <assignment> <COMMA>

<logic_expr> <COMMA> <for_count>

<RP> <LB> <stmt_list> <RB>

<for_count> ::= <arithmetic_op> | <ID><INCREMENT> |

	<ID><DECREMENT>
<while_loop>	::= <WHILE> <LP> <logic_expr> <RP> <LB> <stmt_list> <RB>
<return_stmt>	::= <RETURN> <arithmetic_op> | <RETURN> <NEW_LINE>
<logic_expr>	::= < exprAND><OR_SIGN><logic_expr> | <exprAND>
<exprAND>	::= <exprNOT> <AND_SIGN> <exprAND> | <exprNOT>
<exprNOT>	::= <NOT> <BOOLEAN> | <exprEQ>
<exprEQ>	::=<exprNEQ> <EQUAL_SIGN> <exprEQ> | <exprNEQ>
<exprNEQ>	::= <exprLT><NOTEQUAL_SIGN> <exprNEQ>| <exprLT>
<exprLT>	::= <exprGT> <LESS_SIGN> <exprLT>|<exprGT>
<exprGT>	::= <exprLE> <GREATER_SIGN><exprGT> | <exprLE>
<exprLE>	::= <exprGE> <LESSEQUAL_SIGN><exprLE> | <exprGE>
<exprGE>	::= <arithmetic_op> <GREATEQUAL_SIGN> <exprGE> |<arithmetic_op>
<assignment>	::= <ID> <ASSIGN_OP> <arithmetic_op> |<ID><ASSIGN_OP><function_caller>

|<ID> <ASSIGN_OP><ARRAY_INIT>

|<ID> <ASSIGN_OP><BOOLEAN>

|<ID> <ASSIGN_OP><SWITCH>

|<ID> <ASSIGN_OP><STRING>

<arithmetic_op> ::= <arithmetic_op> <PLUS_SIGN> <exprB>

| <arithmetic_op> <MINUS_SIGN> <exprB>

| <exprB>

<exprB> ::= <exprB> <MULT_SIGN> <exprC>

| <exprB> <DIVID_SIGN> <exprC>

| <exprB> <MOD_SIGN> <exprC>

| <exprC>

<exprC> ::= <exprC> <POW_SIGN> <term> | <term>

<term> ::= <INT> | <DOUBLE> | <ID> | <array_access>

<comment> ::= <START_COMMENT> <STRING>

<END_COMMENT>

<array_access> ::= <ID> <SLB> <arithmetic_op> <SRB>

Functions

<prim_functs> ::= <connect_URL>

| <getTemperature>

| <getHumidity>

| <getAirPressure>

| <getAirQuality>

| <getLight>

| <getSound>

| <getTime>

| <switchOn>

| <switchOff>

| <send>

| <receive>

<connect_URL> ::= <CONNECT_URL><LP> <QUOTATION> <URL>

<QUOTATION><RP>

<getTemperature> ::= <GET_TEMP> <LP> <RP>

<getHumidity> ::= <GET_HUM> <LP> <RP>

<getAirPressure> ::= <GET_AP> <LP> <RP>

<getAirQuality> ::= <GET_AQ> <LP> <RP>

<getLight> ::= <GET_LIGHT> <LP> <RP>

<getSound> ::= <GET_SOUND> <LP> <RP>

<getTime> ::= <GET_TIME> <LP> <RP>

<switchOn> ::= <SWITCH_ON> <LP><SWITCH><RP>

<switchOff> ::= <SWITCH_OFF> <LP><SWITCH><RP>

<send> ::= <SEND_INT> <LP><INT> <TO> <QUOTATION>

<URL> <QUOTATION> | <SEND_INT> <LP> <ID>

<TO> <QUOTATION> <URL> <QUOTATION> <RP>

<receive> ::= <RECEIVE_INT> <LP> <ID> <FROM>

<QUOTATION> <URL> <QUOTATION> <RP>

<print> ::= <PRINT><LP><STRING><RP>

|<PRINT><LP><ID><RP>

2. Description of Nontrivial Tokens

2.1. Comment

In the world of programming, comments help programmers to leave notes as a reminder, a clarification or a warning to the reader. Since it guides readers through the code without disrupting its flow, it increases readability. Comments are enclosed within arrow signs **<* *>** in this language. For example: **<* Check this *>**

2.2. Identifiers

In this language identifiers are called ID shortened form of identifier. Identifiers must start with a lowercase letter followed by a digit, upper case letter, lower case letter, dash (-) or underscore (_).

<ID> : It is used to make sure that the identifier starts with a lowercase letter.

<RESTOFIT> : It is defined recursively to allow ID to include digit, uppercase, lowercase or underscore.

<VAR_WORD> : It is defined recursively to allow var_word to include digit, uppercase or lowercase.

<LOW_ABC> : Lowercase letters. (**a | b | c| d | e | f| g | h | ı | j | k | l | m | n | o | p | r | s | t |u | v | y | z**)

<UP_ABC> : Uppercase letters. (**A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P | R | S | T | U | V | Y | Z**)

<ABC> : Combination of Lowercase and Uppercase alphabetic characters.

<DIGIT> : Digits. (**0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9**)

<ABC_DIGIT> : Combination of Lowercase, Uppercase and Digit. (<ABC> | <DIGIT>)

2.3. Literals

<INT_POS> :

Represents an unsigned integer (without a negative sign). Can be a single digit (0-9) or a sequence of digits (e.g., 123, 456).

<INT_NEG>:

Represents a signed integer with a negative sign (-). Consists of a minus sign

followed by a positive integer.

<INT>:

Represents any integer, including both positive and negative integers. Can be expressed using either <INT_POS> or <INT_NEG>.

<DOUBLE>:

Represents a real number with a decimal point (.). Composed of an integer part (using <INT>), a decimal point, and a fractional part (using <INT_POS>).

2.4. Reserved Words

run : To start the program

if & **else** : To arrange conditionals

for & **while** : To arrange loops

function>> & **return** : To arrange functions

true & **false** : For boolean

getTemperature & **getHumidity** & **getAirPressure** & **getAirQuality** & **getLight** & **getSound** & **connectToURL** & **switchOn** & **switchOff** & **getTime** & **receiveInt** & **sendInt** & **to** & **from :** For functions to serve properly

print : Prints **<STRING>** or **<ID>** to the console

2.5. Operators & Symbols

Punctuation:

<LB>: Left curly brace ("{"), often used for code blocks.

<RB>: Right curly brace ("}"), closing a code block.

<SLB>: Left square bracket ("["), for initialising and accessing arrays.

<SRB>: Right square bracket ("]"), closing an array.

<LP>: Left parenthesis ("("), used in expressions.

<RP>: Right parenthesis (")"), closing an expression.

<COMMA>: Comma (","), separating elements in a list or other contexts.

Comparison Operators:

<EQUAL_SIGN>: Equal sign ("=="), checks for equality.

<NOTEQUAL_SIGN>: Not equal ("!="), checks for inequality.

<LESS_SIGN>: Less than sign ("<"), checks if one value is smaller than another.

<GREAT_SIGN>: Greater than sign (">"), checks if one value is larger than another.

<GREATEQUAL_SIGN>: Greater than or equal sign (">="), checks if one value is greater than or equal to another.

<LESSEQUAL_SIGN>: Less than or equal sign ("<="), checks if one value is less than or equal to another.

Logical Operators:

<OR_SIGN>: OR operator ("|"), represents logical OR.

<AND_SIGN>: AND operator ("&"), represents logical AND.

Arithmetic Operators:

<PLUS_SIGN>: Plus sign ("+"), addition.

<MINUS_SIGN>: Minus sign ("-"), subtraction.

<DIVID_SIGN>: Division sign ("/").

<MULT_SIGN>: Multiplication sign ("*").

<MOD_SIGN>: Modulo operator ("%"), remainder after division.

<POW_SIGN>: Power operator ("^"), exponentiation.

Other Operators and Symbols:

<ASSIGN_OP>: Assignment operator ("<-"), assigns a value to a variable.

<SWITCH>: Keyword representing an alternative decision structure (similar to "switch" in some languages).

<INCREMENT>: Increment operator ("++"), increases a variable by 1.

<DECREMENT>: Decrement operator ("--"), decreases a variable by 1.

<START_COMMENT>: Start of a comment ("<*"), used to annotate code without execution.

<END_COMMENT>: End of a comment ("*>").

<FUNCTION_CALL>: Function call operator (">>"), calls a defined function.

<SEPARATOR>: Separator ("%%") to separate sections for function declaration and statements (main method) of the program.

Special Characters:

<QUOTATION>: Double quotation mark (" ") for strings.

<SYMBOLS>: Various symbols (!, &,\, ?, , . , _, |, #,:,=) used for commenting.

Character Sets:

<LOW_ABC>: Lowercase alphabet (a-z).

<UP_ABC>: Uppercase alphabet (A-Z).

<DIGIT>: Digits (0-9).

3. Description of Nonterminals

<program>:

It is the beginning of the program. It starts with the keyword “run” . After the run keyword we have used two separate sections for function declaration and statements (main method) of the program. In order to separate these two sections a SEPARATOR (%%) is used. In curly brackets the user will type the main method, after the separator user will be able to define their own functions.

<stm_list>:

stm_list stands for “statement list”. It involves the statements of the main method within.

<stmt>:

stmt stands for “statement”. We have divided all statements of our language into two parts. <matched> and <unmatched> statements.

<matched>:

Matched non-terminal represents this language's matched statements which means either an if-else statement which are if’s that are associated with an else or

any other statement other than if which are not associated with any else. If a statement is matched it uses the keyword **if**. Every other statement such as **while**, **for**, assignment, arithmetic etc. are considered as unconditional statements. There can be multiple statements inside an if, else or loop blocks so that there is freedom to have complex porgrams.

<unmatched>:

This non-terminal represents unmatched if statements only. Which are if statements that don't have an associated matched **else** keyword. It uses the **if** keyword.

<nonconditional_stmt>:

This non-terminal represents the non conditional statements which are every single statement other than if statements. It includes <prim_functs>, <comment> , <assignment>, <arithmetic_op> , <loops>, <return_stmt>, <function_caller>, <print>.

<prim_functs>:

These are built-in functions provided by the language, allowing you to interact with sensors, retrieve data, control devices, and more. Prim functions include <connect_URL>, <getTemperature>, <getHumidity>, <getAirPressure> ,<getAirQuality>, <getLight>, <getSound> <getTime>, <switchOn>, <switchOff>, <send>, <receive>.

<connect_URL>:

Establishes a connection to a specified URL, allowing you to interact with web services or online resources. It uses **connectURL** keyword followed by parentheses without parameters.

<getTemperature>:

The predefined function to reach the data of the sensor detects the temperature of the environment. It uses **getTemperature** keyword keyword followed by parentheses without parameters.

<getHumidity>:

The predefined function to reach the data of the sensor detects the humidity of the environment. It uses **getHumidity** keyword keyword followed by parentheses without parameters.

<getAirPressure>:

The predefined function to reach the data of the sensor detects the air pressure of the environment. It uses **getAirPressure** keyword keyword followed by parentheses without parameters.

<getAirQuality>:

The predefined function to reach the data of the sensor detects the air pressure of the environment. It uses **getAirQuality** keyword keyword followed by parentheses without parameters.

<getLight>:

The predefined function to reach the data of the sensor detects the air light of the environment. It uses **getLight** keyword keyword followed by parentheses without parameters.

<getSound>:

The predefined function to reach the data of the sensor detects the sound level of the environment. It uses **getSound** keyword keyword followed by parentheses without parameters.

<getTime>:

The predefined function to reach the time data of the system. It uses the keyword **getTime** keyword followed by parentheses without parameters. It returns an array that saves six integers as the current time: year, month, day, hour, minute, second.

<switchOn>:

The predefined function that uses **switchOn** keyword followed by parentheses and switch identifier within parentheses. Used to control demanded switches.

<switchOff>:

The predefined function that uses **switchOff** keyword followed by parentheses and switch identifier within parentheses. Used to control demanded switches.

<send>:

The predefined function that uses **sendInt** keyword followed by parentheses. Parameter this function takes is an integer that will be sent to the currently connected server.

<receive>:

The predefined function that uses **receiveInt** keyword followed by parentheses. Parameter this function takes is an integer that will be received from the currently connected server.

<comment> :

This nonterminal represents how to write comments in this language. Comment text <STRING> is placed between "<* " and "*>" .

<assignment> :

It allows users to declare assignments, variables can be assigned to arithmetic operations, variables returned by statements, arrays, booleans, switch names, strings. This nonterminal provides variable manipulation through various mechanisms. Expressions can be assigned like calculations or sensor readings, strings directly store function return values, create and initialise arrays, assign boolean flags true, false.

<arithmetic_op>:

This nonterminal represents arithmetic operations. The token <arithmetic_op> is used for plus and minus sign, the token <exprB> is used for multiplication, division and modular sign, <exprC> is used for power sign and <term> as the leaf of this arithmetic operation parse tree. In order to reach correct associativities for these operations and precedence of each.

<array_access>:

It declares how to access an element of an array. It is used by the identifier of the

array and the index of the element which can be represented by an arithmetic operation or a term within square brackets.

<function_declarations>:

This is one of the main parts of the **<program>**. This token designates the area within a program where user-defined functions are declared. It may be empty if no custom functions are needed, or it can contain a series of function definitions. Its purpose is to provide an allocated space for constructing reusable code blocks.

<function_declaration>:

This token outlines the structure of each individual user-defined function. It begins with the keyword **function**, followed by a unique name (identifier) for the function. Parentheses enclose an optional parameter list, specifying the types of input values the function can receive. Curly braces encompass the function body, containing the statements that define the function's operations.

<function_caller>:

This nonterminal defines how to call predefined or user defined functions within program. >> operator is used in COSTANZA in order to call a function with parentheses that has a parameter list within.It is invoked with the keyword **<FUNCTION_CALL>** which is lexeme "**>>**", followed by the exact name (identifier) of the function to be called. Parentheses surround an optional argument list, providing the values to be used by the function during its execution if needed.

<param_list>:

This token specifies the optional list of parameters a user-defined function can accept. It can be empty, meaning the function does not require any input, or it can contain one or more parameters. Each parameter within the list is represented by an <ID>, which acts as the name or identifier for the expected input value. When multiple parameters are present, they are separated by commas.

<loops>:

This token includes two fundamental control flow structures used for repeated

execution of code blocks: **<for_loop>** and **<while_loop>**. They enable the program to efficiently perform tasks multiple times based on specified conditions.

<for-loop>:

This token represents a loop that iterates a specific number of times based on conditions. It starts with the keyword **for** followed by a control expression within parentheses. This expression initialises a loop counter variable, specifies a continuation condition, and updates the counter on each iteration. The loop executes the statements within curly braces as long as the condition remains true. The counter can be modified using arithmetic operations, incremented, or decremented during each iteration.

<for-count>:

This token determines how the loop counter variable changes within a <for_loop>. It can take three forms: <arithmetic_op>: This allows you to modify the loop counter's value using any valid arithmetic expression on each iteration, <ID><INCREMENT>: This shorthand increments the loop counter variable by 1 after each iteration, <ID><DECREMENT>: This shorthand decrements the loop counter variable by 1 after each iteration.

<while-loop>:

This token represents a loop that continues executing code as long as a specified condition holds true. It begins with the keyword **while**, followed by the loop's condition between parentheses. The loop's body, containing the code to be repeated, is written between curly braces. As long as the conditions are true, the statements within the loop body are executed repeatedly.

<return_stmt>:

The <return_stmt> token dictates how a function delivers a value back to the calling function. It has two distinct forms. Either **return <arithmetic_op>**, which returns the value of the expression **<arithmetic_op>**, **return <NEW_LINE>** returns no value. Both forms should be used within the body of a user-defined function.

<logic_expr>:

This nonterminal represents logical expressions, forming the conditions that control program flow within loops and conditional statements. It allows for complex conditions to be built using a combination of logical operators and simpler expressions.

- **< exprAND><OR_SIGN><logic_expr**>: Combines two logical expressions using the logical OR operator (||), resulting in true if either individual expression is true.
- **<exprNOT> <AND_SIGN> <exprAND>:** Combines two logical expressions using the logical AND operator (&&), resulting in true only if both individual expressions are true.
- **<exprNOT>:** Represents a negated expression, achieved by placing the **not** keyword before another <exprNOT>, a boolean value (<BOOLEAN>), or an equality expression (<exprEQ>).
- **<exprEQ>:** Represents an equality comparison between two expressions. It uses comparison operators like <EQUAL_SIGN> (==) to check if two expressions are equal, and also includes <NOTEQUAL_SIGN> (**not** =) for checking inequality.
- **<exprNEQ>:** Represents expressions involving inequality comparisons. It uses operators like <NOTEQUAL_SIGN> (**not** =) to check if two expressions are not equal, and also includes <LESS_SIGN> (<) for less than comparisons.
- **<exprLT>**: Represents expressions involving comparisons of "less than" relationships. It uses operators like <LESS_SIGN> (<) to check if one expression is less than another.
- **<exprGT>**: Represents expressions involving comparisons of "greater than" relationships. It uses operators like <GREATER_SIGN> (>) to check if one expression is less than another.
- **<LESSEQUAL_SIGN>** : Represents expressions involving comparisons of "less or equal than" relationships. It uses operators like <LESSEQUAL_SIGN> (>=) to check if one expression is less or equal to

another.

- **<GREATEQUAL_SIGN>** : Represents expressions involving comparisons of "greater or equal than" relationships. It uses operators like <GREATEQUAL_SIGN> (**>=**) to check if one expression is greater or equal to another.

<assignment>:

This nonterminal defines how variables are assigned to values within the program. It allows various ways to store data in variables, **<-** is used as assignment operator.

- **<ID> <ASSIGN_OP> <arithmetic_op>:** Assigns the value of an arithmetic operation <arithmetic_op> to a variable identified by <ID>.
- **<ID> <ASSIGN_OP> <function_caller>:** Assigns the return value of a function call <function_caller> to a variable identified by <ID>.
- **<ID> <ASSIGN_OP> <ARRAY_INIT>:** Assigns initial values to an array variable using an array initializer <ARRAY_INIT>.
- **<ID> <ASSIGN_OP> <BOOLEAN>:** Assigns a boolean value (<BOOLEAN>) to a variable identified by <ID>.
- **<ID> <ASSIGN_OP> <SWITCH>:** Assigns a switch (<SWITCH>) to a variable identified by <ID>
- **<ID> <ASSIGN_OP> <STRING>:** Assigns a string (<STRING>) to a variable identified by <ID>

<arithmetic_op>:

This nonterminal defines the structure of arithmetic expressions, provides calculations involving addition, subtraction, multiplication, division, and exponentiation in a recursive manner in order to provide needed associativity to the arithmetic operations:

- **<arithmetic_op> <PLUS_SIGN> <exprB>:** Adds the value of <exprB> to the current value of the arithmetic expression.
- **<arithmetic_op> <MINUS_SIGN> <exprB>:** Subtracts the

value of <exprB> from the current value of the arithmetic expression.

- **<exprB>:**Represents the result of an expression involving multiplication, division, or modulo operation.

<exprB>:

This nonterminal defines the structure of expressions involving multiplication, division, and modulo operations:

- **<exprB> <MULT_SIGN> <exprC>:** Multiplies the value of <exprC> by the current value of the expression.
- **<exprB> <DIVID_SIGN> <exprC>:** Divides the current value of the expression by the value of <exprC>.
- **<exprB> <MOD_SIGN> <exprC>:** Performs the modulo operation, returning the remainder when the current value of the expression is divided by the value of <exprC>.
- **<exprC>:** The result of an expression involving exponentiation.

<exprC>:

This nonterminal defines expressions involving exponentiation:

- **<exprC> <POW_SIGN> <term>:** Calculates the current value of the expression to the power of <term>.

<term>:

Represents an integer (<INT>) or a double-precision floating-point number (<DOUBLE>) or an ID <ID>, or an array_access <array_access>.

<print>:

This nonterminal defines how to print a message to console.

<comment>:

This nonterminal defines how comments are included in the code for better readability and documentation. Comments are enclosed between

4. Language Criteria

4.1 Readability:

Syntax Familiarity:

- Costanza adopts a syntax similar to classical imperative languages, enhancing readability for users familiar with programming.
- Keywords like “if”, “else”, “for”, “while”, and “return” are used, providing a recognizable structure to control program flow.
- The use of common operators such as arithmetic (**+, -, *, /, %**) and logical (**&, |, !**) further aligns with standard programming conventions, making the language intuitive to understand.

Structured Blocks:

- Block statements like if, else, for, and while are clearly defined with opening and closing curly braces ({}), ensuring clarity in code organisation.
- The consistent use of indentation within blocks enhances readability by visually representing the program's structure.

Descriptive Function Names:

- Built-in functions are intuitively named to reflect their purpose, such as “getTemperature”, “switchOn”, and “sendInt”, contributing to code clarity and comprehension.
- User-defined functions can also be named descriptively, enabling developers to create meaningful abstractions for reusable code segments.

Comments:

- Costanza allows developers to include comments using angle brackets and asterix (**<*...*>**), providing a way to document code and explain complex logic.
- Well-commented code segments can significantly enhance readability, especially for non-expert programmers or those unfamiliar with the specific implementation details.

Consistent Naming Conventions:

- Identifiers follow consistent naming conventions, starting with a

lowercase letter and allowing alphanumeric characters, underscores, and dashes.

- Clear and consistent naming conventions reduce cognitive overhead and facilitate easier comprehension of variable and function names.

3.2 Writability:

Expressive Syntax:

- Costanza's syntax prioritises expressiveness, allowing developers to articulate their intentions clearly and concisely.
- The absence of strict type checking reduces verbosity, enabling developers to focus on solving problems without being encumbered by explicit type declarations.

Top-Down Approach:

- The language adopts a top-down approach, facilitating rapid development of small to medium-sized programs suitable for controlling IoT devices.
- By providing built-in functions for common IoT operations and supporting custom function definitions, developers can efficiently organise and manage their code.

Flexible Control Structures:

- Flexible control structures like for and while loops offer versatility in designing algorithms for IoT applications, accommodating various iteration patterns and conditions.

Readable Function Calls:

- Function calls in Costanza utilise a clear syntax with the >> operator, followed by the function name and optional parameters enclosed in parentheses.
- This concise syntax enhances writability by reducing the cognitive load associated with function invocation.

Semicolon-Free Statements:

- Costanza's decision to omit semicolons at the end of statements contributes to a more fluid and natural coding experience.
- The absence of semicolons eliminates unnecessary punctuation, streamlining the writing process and reducing the potential for syntax

errors.

3.3 Reliability:

Error Prevention Mechanisms:

- While Costanza forgoes strict type checking to prioritise writability, it incorporates error prevention mechanisms to maintain reliability.
- The language's BNF grammar and lexical rules are designed to minimise ambiguity and syntactic errors, promoting consistent and predictable behaviour during program execution.

Robust Standard Library:

- The inclusion of a robust standard library with built-in functions tailored to IoT operations enhances the reliability of Costanza programs.
- Standardised functions for sensor data retrieval, device control, and network communication mitigate the risk of errors and inconsistencies in IoT applications.

By prioritising readability, writability, and reliability, Costanza aims to provide an accessible and effective programming language tailored for IoT development. Its intuitive syntax, expressive features, and robust ecosystem contribute to a positive development experience and facilitate the creation of reliable IoT applications.